

AI Shopping Assistant

System Design & Architecture

A practical overview of the architecture, data flow, API design, AI integration, security, and production deployment.

Frontend React + Vite + JavaScript + Tailwind CSS	Backend Python + Django REST Framework
Database PostgreSQL	AI Gemini API
Deployment Vercel + Render	3D / UI React Three Fiber + Three.js + Drei

Project documentation • [GitHub reference](#)

1. Project Overview

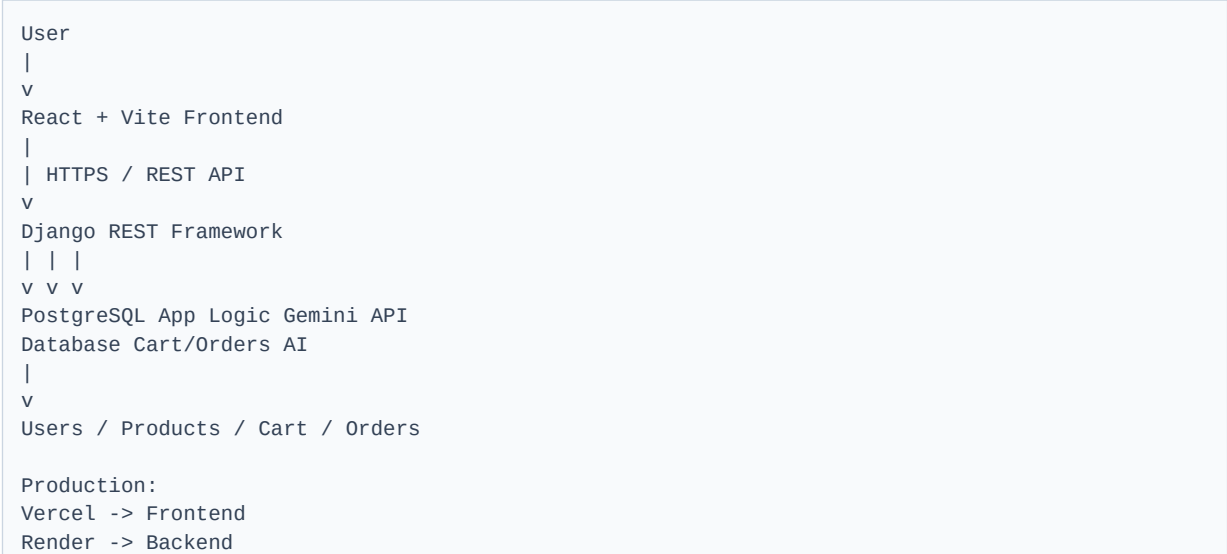
AI Shopping Assistant is a full-stack e-commerce application built around a normal shopping workflow with an additional natural-language interface. Users can create an account, browse products, view details, manage a cart, complete checkout, and view previous orders.

The AI assistant is connected to the application itself. It can understand shopping requests and, for supported requests, trigger a real backend action such as adding a matched product to the user's cart.

The goal of the design is to keep the core application predictable and reliable while making AI an additional way to interact with it.

2. System Architecture

The application is split into a frontend, backend, database, and AI service. Each part has a clear responsibility.



Django remains the source of truth for application data. Gemini helps interpret natural-language requests but does not directly own database state.

3. Main Components

Component	Responsibility
React + Vite	UI, routing, product pages, cart, checkout, orders, and the AI assistant interface.
Django REST Framework	REST APIs, authentication, product access, cart/order logic, and AI orchestration.
PostgreSQL	Persistent application data including users, products, cart data, and orders.
Gemini API	Natural-language understanding for shopping requests and supported AI actions.
Vercel	Production hosting for the React frontend.
Render	Production hosting for the Django backend.
Three.js / React Three Fiber / Drei	Interactive 3D elements in the interface.

4. AI Shopping Assistant Flow

The AI assistant is implemented as part of the application flow rather than as a separate chatbot page. The frontend sends the message to Django, the backend communicates with Gemini, and supported actions are passed through

the application's existing business logic.

```
User
|
v
ShoppingAssistant (React)
|
| POST /api/ai/chat/
v
Django AI endpoint
|
v
Gemini API
|
+-----+
| |
v v
Normal reply Supported action
|
v
POST /api/ai/cart/add/
|
v
Django cart logic
|
v
PostgreSQL
|
v
Updated cart
```

Example: “Add the women's kurti to my cart” → understand the request → identify the product → perform the cart operation → refresh the cart.

5. API Design

The frontend communicates with Django through REST endpoints. The API boundary keeps presentation separate from application logic.

Representative endpoints

Endpoint	Method	Purpose
/api/products/	GET	Return the product catalog.
/api/ai/chat/	POST	Process a natural-language shopping request.
/api/ai/cart/add/	POST	Add a product selected by a supported AI action to the user's cart.
Cart APIs	GET/POST/PUT/DELETE	Read and modify the authenticated user's cart.
Checkout / Orders APIs	POST/GET	Create orders and retrieve order history.

Example AI request

```
POST /api/ai/chat/

{
  "message": "Add the women's kurti to my cart"
}
```

The backend processes the request and, for a supported action, uses the authenticated user's application state to perform it.

The Gemini API key is kept on the backend. The browser communicates with Django rather than calling Gemini directly.

6. Database Design

PostgreSQL stores the persistent state of the shopping application. The main relationships are centered around users, products, carts, and orders.

```
User
|
+---- Cart ---- CartItem ---- Product
|
+---- Order --- OrderItem --- Product

User -> owns user-specific shopping data
Product -> represents an item in the catalog
Cart -> represents the current shopping state
CartItem -> connects a cart with a product and quantity
Order -> represents a completed purchase
OrderItem -> records products included in an order
```

The browser and AI response are not treated as the final source of truth. Cart and order state is persisted and validated through the backend.

7. Authentication Flow

```
User
|
v
Login / Signup
|
v
Django authentication
|
v
Access token
|
v
React stores authentication state
|
v
Protected API request
|
| Authorization: Bearer <token>
v
Django validates user
|
v
User-specific cart / checkout / orders
```

Protected application areas use the authenticated user's identity when accessing user-specific data. This is especially important for cart, checkout, and order operations.

8. Security & Configuration

- Production secrets such as the Django secret key and Gemini API key are supplied through environment variables and are not committed to the repository.
- The backend is configured to understand HTTPS behind the production proxy.
- Secure session and CSRF cookies are enabled when DEBUG is disabled.
- CORS is restricted to configured frontend origins.
- Application actions are performed by Django rather than trusting client-side or AI-generated state.

- The AI service is behind the backend, keeping the provider credential out of the browser.

Django deployment checks were used during the production security cleanup. HSTS was deliberately left out rather than enabling it without a confirmed production policy.

9. Production Deployment

```
Internet
|
+-----+-----+
| |
v v
Vercel Render
React frontend Django REST API
| |
+----- HTTPS -----+
|
+-----+-----+
| |
v v
PostgreSQL Gemini API
data AI
```

The frontend and backend are deployed separately. The frontend receives the backend URL through environment configuration, while backend secrets are configured in the hosting environment.

Current deployment uses Vercel for the frontend and Render for the Django backend.

10. Core Application Flows

Authentication

Signup / Login -> Django authentication -> token state -> protected routes

Shopping

Product catalog -> Product details -> Add to cart -> Cart -> Checkout -> Order

AI-assisted shopping

Natural-language request -> AI interpretation -> supported backend action -> database -> updated UI

Order history

Authenticated user -> Orders API -> stored order data -> Orders page

11. Reliability & Performance

- The frontend uses staged loading messages so users get immediate feedback while the backend request is completing.
- Normal browsing, cart, checkout, and order operations do not require an AI request.
- The frontend and backend are independently deployable.
- PostgreSQL provides persistent state across requests.
- The current Render free instance can spin down after inactivity, which may make the first request slower after a period without traffic.

12. Basic Failure Handling

The application keeps external services behind controlled application boundaries. This makes failures easier to handle without exposing internal service details to the user.

```

Gemini unavailable
|
v
Django catches AI/API failure
|
v
Controlled response to frontend
|
v
User sees an error / fallback message

Database/API failure
|
v
Django returns an appropriate API error
|
v
Frontend displays its error state

```

This separation prevents a third-party AI or database failure from becoming an uncontrolled browser-side failure.

13. Design Decisions

Decision	Reason
AI behind Django	Keeps the AI credential private and centralizes AI-related logic.
Backend owns application state	Prevents the browser or model from becoming the source of truth.
REST API boundary	Keeps frontend and backend responsibilities separate and reusable.
PostgreSQL	Fits the relational nature of users, products, carts, and orders.
Separate Vercel/Render deployment	Allows the frontend and backend to be deployed and debugged independently.
Environment-based secrets	Keeps credentials out of source control.

14. Scaling Path

The current architecture is intentionally simple for the project's present scale. If traffic grows, the same separation allows individual parts to evolve without rewriting the application.

```

More traffic
|
v
Load balancer
|
+-----+-----+
| | |
v v v
Django Django Django
instance instance instance
|
v
Managed PostgreSQL
|
+--> Cache when useful
|
+--> Async jobs when useful
|
+--> AI service

```

Scaling components should be introduced when actual workload requires them. The project does not depend on microservices or a large cloud architecture just for the sake of complexity.

15. Future Improvements

- Product search, filtering, and sorting for larger catalogs.
- More personalized AI recommendations using user preferences and product history.
- Automated frontend and backend tests with CI.
- Online payment integration and order tracking.
- Application monitoring and performance metrics.
- Caching or asynchronous processing where measurable traffic makes it useful.
- A managed production infrastructure tier when availability or traffic requirements justify it.

Project: AI Shopping Assistant **Author:** Amrutha BM